# 💻 Clase 7 – Programación Orientada a Objetos (Versión Completa)

**Docentes:**

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano

---

## 🎯 Objetivos

- Introducir el paradigma de **Programación Orientada a Objetos (POO)**.
- Comprender los conceptos fundamentales: **clase**, **objeto**, **atributo**, **método**.
- Relacionar la POO con la programación estructurada vista en clases anteriores.
- Analizar ejemplos en **C++** y **Java**.

---

## 🧩 Introducción

La **Programación Orientada a Objetos (POO)** surge como una evolución del paradigma estructurado. Busca **representar entidades del mundo real** dentro del programa.

**Idea central:**

> Un programa orientado a objetos es un conjunto de objetos que colaboran entre sí para cumplir un objetivo.

---

## Programación Orientada a Objetos (POO)

> El paradigma orientado a objetos es útil cuando el sistema se modela de forma casi análoga a la realidad, porque así se simplifica el diseño de alto nivel. Esta analogía permite que los programadores tengan más claro cuál es el papel de cada porción del programa y de los datos, lo que facilita la creación y el mantenimiento del sistema. Además, se promueve la reutilización, pues las similitudes entre objetos se programan sólo una vez en forma abstracta y el programador concentra su esfuerzo en las diferencias concretas.

---

## 🧱 Conceptos Fundamentales

| Concepto | Descripción |
|---|---|
| **Clase** | Plantilla o molde que define las características y comportamientos de un tipo de objeto. |
| **Objeto** | Instancia concreta de una clase. |
| **Atributo** | Propiedad o dato que describe al objeto. |
| **Método** | Acción o comportamiento que el objeto puede realizar. |

---

### Diagrama de Clase (modelo conceptual)

```
+--------------------+
|     Persona        |
+--------------------+
| - nombre: string   |
| - edad: int        |
+--------------------+
| + saludar()        |
| + cumplirAnios()   |
+--------------------+
```

*El diagrama muestra una clase con sus atributos y métodos.*

---

## Clase y Objeto en C++ (según diapositivas)

```cpp
#include <iostream>
using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    void saludar() {
        cout << "Hola, soy " << nombre << endl;
    }
};

int main() {
    Persona p1;
    p1.nombre = "Juan";
    p1.edad = 25;
    p1.saludar();

    return 0;
}
```

*Ejemplo tomado del material de clase 7: muestra la creación de una clase, la instancia de un objeto y el uso de un método.*

---

## Ejemplo en Java (según diapositivas)

```java
public class Persona {
    String nombre;
```

```
    int edad;

    void saludar() {
        System.out.println("Hola, soy " + nombre);
    }

    public static void main(String[] args) {
        Persona p1 = new Persona();
        p1.nombre = "Juan";
        p1.edad = 25;
        p1.saludar();
    }
}
```

*Ejemplo equivalente al de C++, con sintaxis Java.*

---

## Concepto de Instancia

- Una **instancia** es un objeto creado a partir de una clase.
- Cada instancia tiene sus propios valores para los atributos definidos en la clase.
- Pueden existir múltiples objetos del mismo tipo.

**Ejemplo visual:**

```
Clase Persona
   |
   ├──> Objeto 1: nombre = "Ana", edad = 30
   ├──> Objeto 2: nombre = "Luis", edad = 22
   └──> Objeto 3: nombre = "Juan", edad = 25
```

---

## Comparativa entre paradigmas

| Aspecto | Programación Estructurada | Programación Orientada a Objetos |
|---|---|---|
| **Unidad básica** | Función o procedimiento | Clase / Objeto |
| **Datos** | Variables y estructuras globales | Atributos de clase |
| **Acciones** | Secuencia de instrucciones | Métodos de los objetos |
| **Control** | Flujo lineal | Mensajes entre objetos |
| **Enfoque** | Algorítmico | Conceptual (modelado del mundo real) |

---

## Encapsulamiento

El **encapsulamiento** consiste en **ocultar los datos internos de un objeto** y exponer solo lo necesario.

```
class Cuenta {
private:
    float saldo;

public:
    void depositar(float monto) {
        saldo += monto;
    }

    float obtenerSaldo() {
        return saldo;
    }
};
```

*Permite proteger la información interna de modificaciones indebidas.*

---

## Herencia

La **herencia** permite crear nuevas clases a partir de otras, reutilizando código.

```
class Persona {
public:
    string nombre;
    int edad;
};

class Estudiante : public Persona {
public:
    string carrera;
};
```

*La clase `Estudiante` hereda los atributos y métodos de `Persona`.*

---

## Polimorfismo

El **polimorfismo** permite que una misma operación se comporte de manera diferente según el objeto que la ejecute.

```
class Animal {
public:
    virtual void hacerSonido() {
        cout << "Sonido genérico" << endl;
    }
};

class Perro : public Animal {
```

```
public:
    void hacerSonido() override {
        cout << "Guau!" << endl;
    }
};

class Gato : public Animal {
public:
    void hacerSonido() override {
        cout << "Miau!" << endl;
    }
};

int main() {
    Animal* a;
    Perro p;
    Gato g;

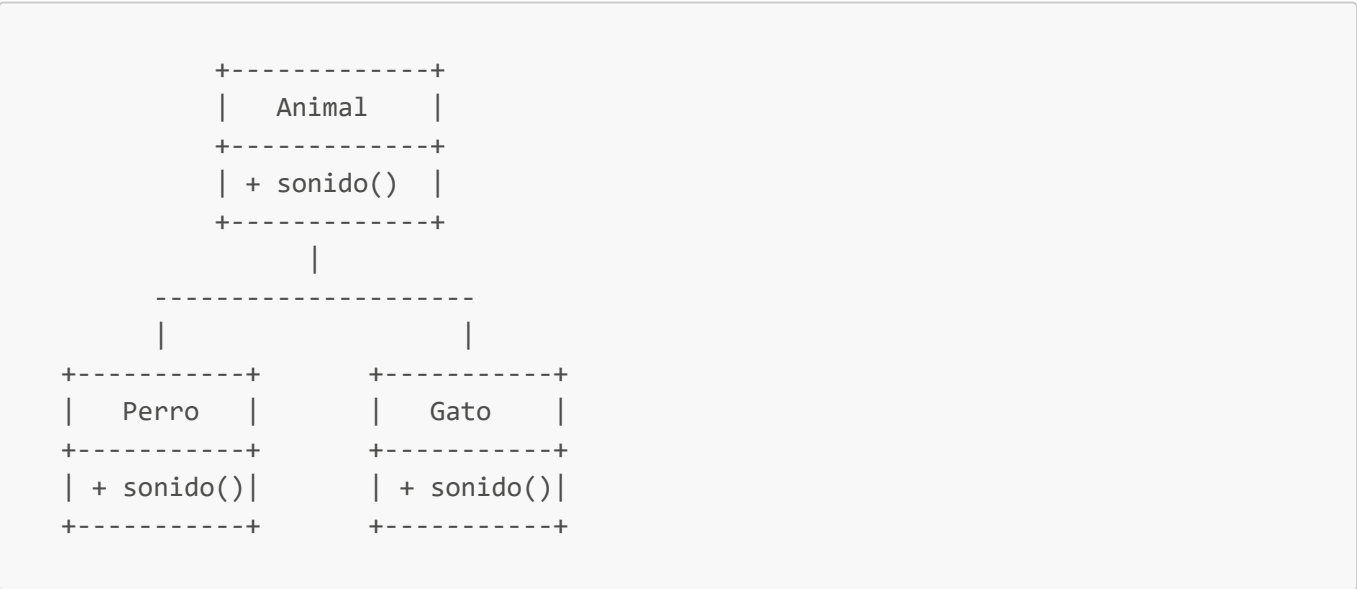
    a = &p;
    a->hacerSonido();  // Guau!

    a = &g;
    a->hacerSonido();  // Miau!
}
```

*Ejemplo de polimorfismo con clases derivadas que redefinen métodos.*

---

## Diagrama ASCII de jerarquía de clases




---

## Conclusión

- La POO permite modelar sistemas complejos de manera más natural.
- Cada objeto tiene identidad, estado (atributos) y comportamiento (métodos).
- Los principios clave son: **abstracción, encapsulamiento, herencia y polimorfismo.**

- Lenguajes como **C++**, **Java** y **Python** adoptan plenamente este paradigma.

---

## Bibliografía


- *El lenguaje C++: Programación Orientada a Objetos* — Balestrini & Di Stefano
- *Programación en Java: Fundamentos y Aplicaciones* — Marzal & Gracia
- *Conceptos de Programación Orientada a Objetos* — Booch, Grady
- *Documentación oficial de Java y C++ (Oracle y ISO)*


---